Travel in London

Stochastic navigation with a fully observable decision-making agent
Practical project | Java 17+ | SPbPU

The application moves an agent over the supplied London schematic until it reaches Tower Hill. A known map and observed position do not make movement deterministic: intended actions can slip sideways. The program computes a policy that minimizes the expected number of attempted moves, then demonstrates the resulting behavior in a Java Swing interface.

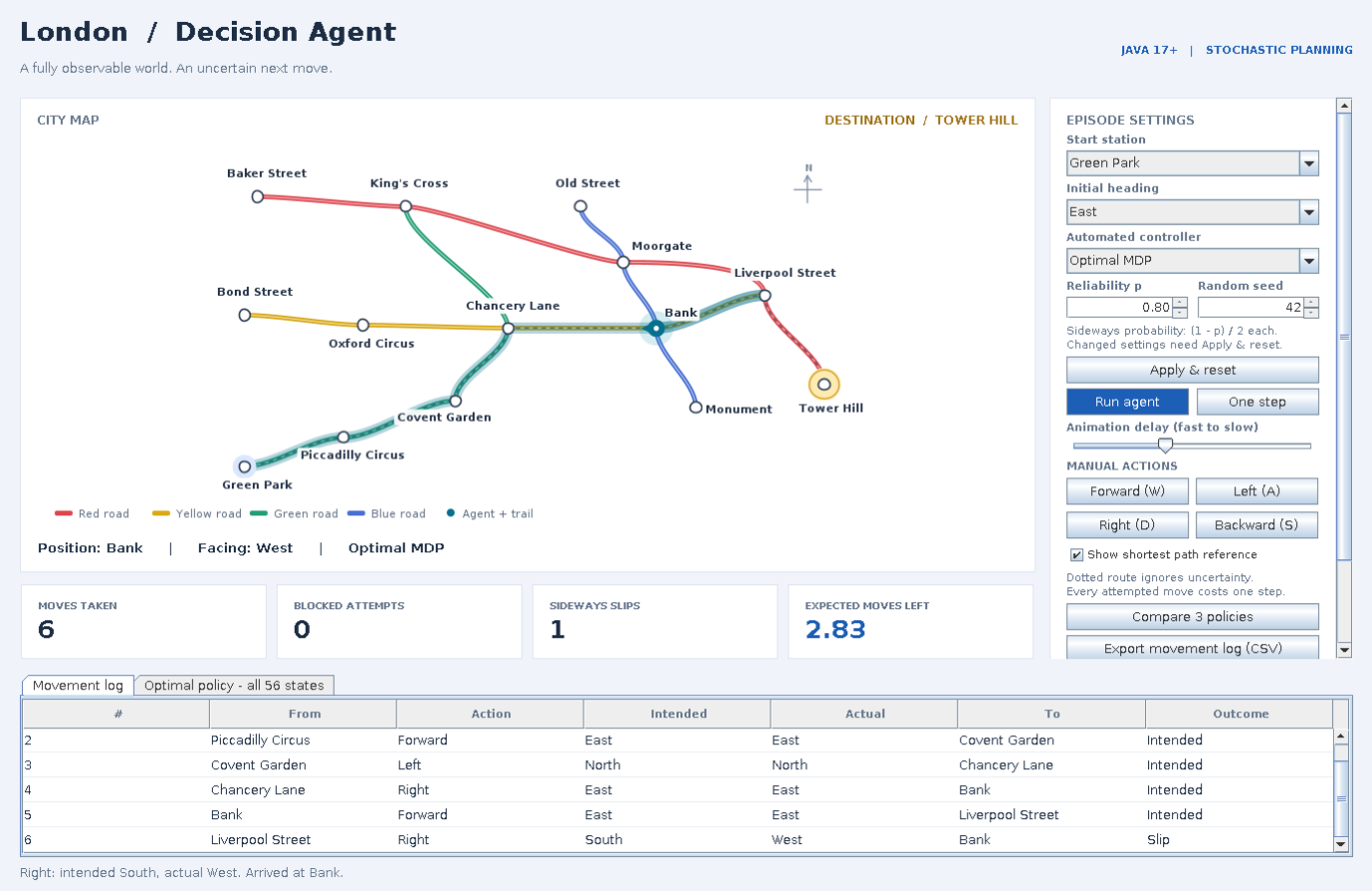


Figure 1. The actual Swing interface after six actions with p=0.80 and seed 42. A sideways error has returned the agent from Liverpool Street to Bank. The next decision uses this new state.

Assignment requirement	Delivered behavior
Map and observed position	14 stations, 15 roads; station and heading are displayed.
Four movement actions	Forward, Backward, Left and Right are relative to heading.
Unreliable movement	Configurable intended probability p and two sideways outcomes.
Single terminal target	Arrival at Tower Hill stops the episode; starting there costs zero.
Optimal decisions	Value iteration for an undiscounted stochastic shortest path model.

Modeling boundary. The assignment does not specify probabilities, costs, starting state, road bearings or a required algorithm. The choices on the next page are explicit implementation assumptions, and the reported results apply to that model.

1. State and motion model

State: s=(i,h), where i is a station and h is North, East, South or West. The 14 stations give 56 representations: 52 nonterminal states and four terminal heading states at Tower Hill. The state and map are fully observed. Tower in the assignment text is identified with Tower Hill on its diagram.

Relative actions: Forward leaves the bearing unchanged; Left turns it 90 degrees counterclockwise; Right turns it 90 degrees clockwise; Backward reverses it. Turning and attempting one road form a single action. The default initial state is Green Park / East, with random seed 42.

```
Intended bearing d = turn(h, action)
P(actual = d)           = p
P(actual = left(d))     = (1-p)/2
P(actual = right(d))    = (1-p)/2
```

The default p=0.80 gives probabilities 0.80, 0.10 and 0.10. If no road exists in the actual bearing, the position stays unchanged and the agent faces that bearing. Otherwise it traverses the road and faces away from the entry port at the destination. This supports curved roads without assigning two roads to one outgoing port.

Reconstructed bidirectional roads

Color	Endpoint A	Port at A	Endpoint B	Port at B
Red	Baker Street	East	King's Cross	West
Red	King's Cross	East	Moorgate	West
Red	Moorgate	East	Liverpool Street	North
Red	Liverpool Street	South	Tower Hill	North
Yellow	Bond Street	East	Oxford Circus	West
Yellow	Oxford Circus	East	Chancery Lane	West
Yellow	Chancery Lane	East	Bank	West
Yellow	Bank	East	Liverpool Street	West
Green	Green Park	East	Piccadilly Circus	West
Green	Piccadilly Circus	East	Covent Garden	West
Green	Covent Garden	North	Chancery Lane	South
Green	Chancery Lane	North	King's Cross	South
Blue	Old Street	South	Moorgate	North
Blue	Moorgate	South	Bank	North
Blue	Bank	South	Monument	North

Interchange symbols are merged. Ports are a discrete convention for this schematic, not measured geographic bearings. The red Moorgate-Liverpool road curves into the north port at Liverpool, while the yellow Bank-Liverpool road enters its west port. Colors have no effect on cost. No real travel times or distances are assumed.

Costs and termination: every attempted move costs one, including a blocked attempt. Terminal states are absorbing with zero further cost. Missing roads are boundaries, not exits; Tower Hill is the sole exit target. GUI episodes have a 500-attempt cap; batch comparisons use 5,000 and explicitly count capped episodes as failures.

2. Planning and implementation

Bellman update

```
J(goal) = 0
J(s) = min_a [ 1 + sum_next P(next | s,a) J(next) ]
policy(s) = argmin_a [ 1 + sum_next P(next | s,a) J(next) ]
```

J(s) is the minimum expected number of attempts until arrival. Synchronous value iteration starts at zero and stops when the largest update is below 10^-10. The final policy is extracted from the converged values; ties follow Forward, Left, Right, Backward. A 100,000-sweep limit raises an error if convergence fails. At p=0.80 the solver needs 47 sweeps and has Bellman residual 3.31 x 10^-11.

This is an undiscounted stochastic shortest path problem, rather than discounted reward maximization. The finite model has a proper goal-reaching policy and strictly positive nonterminal costs. Policies that can fail to reach the goal forever have infinite expected cost. Independent policy evaluation detects such states and solves (I-P_policy)J=1 for finite-cost states using pivoted Gaussian elimination [2].

Known transition probabilities permit model-based planning without training episodes. Heading determines relative action labels; because every bearing is selectable in one attempt without extra turning cost, optimal values are equal across the four headings at the same station.

Component	Java responsibility
CityMap / State / Action	Road ports, observed state, relative directions, BFS.
NavigationModel	Exact outcomes, sampling, blocked movement and arrival heading.
ValueIteration / PolicyEvaluation	Optimal Bellman policy and independent expected-cost check.
Simulation / Csv	Reproducible episodes, statistics and machine-readable results.
LondonPanel / MapPanel	Swing controls, vector map, logs and policy table.
Main / SelfTest	Desktop and CLI entry points; executable verification.

A reproducible episode

#	From	Action	Intended / actual	To
1	Green Park	Forward	East / East	Piccadilly Circus
2	Piccadilly Circus	Forward	East / East	Covent Garden
3	Covent Garden	Left	North / North	Chancery Lane
4	Chancery Lane	Right	East / East	Bank
5	Bank	Forward	East / East	Liverpool Street
6	Liverpool Street	Right	South / West	Bank
7	Bank	Backward	East / East	Liverpool Street
8	Liverpool Street	Right	South / South	Tower Hill

Table 2. Optimal controller, Green Park / East, p=0.80, seed 42. Step 6 slips west instead of moving south. The episode finishes in 8 attempts, with one slip and no blocked attempts. Resetting with the same seed and action sequence reproduces it.

3. Numerical experiments

Each of the nine configurations runs 10,000 episodes from Green Park / East. The base seed is 42 and the cap is 5,000 attempts. Per-episode seeds use a SplitMix64 mixing function before initializing java.util.Random. The policies share this seed schedule; their random-number use can diverge after different decisions. All 90,000 recorded episodes reached the goal.

p	Controller	Mean attempts	SE	Exact mean	P95
0.40	Optimal MDP	22.7665	0.1131	22.7396	44
0.40	BFS	24.4684	0.1277	24.3281	49
0.40	Random	117.7075	0.9548	118.9091	304
0.80	Optimal MDP	8.1854	0.0221	8.1465	12
0.80	BFS	8.1854	0.0221	8.1465	12
0.80	Random	117.6039	0.9536	118.9091	308
1.00	Optimal MDP	6.0000	0.0000	6.0000	6
1.00	BFS	6.0000	0.0000	6.0000	6
1.00	Random	118.6618	0.9746	118.9091	311

SE is the sample standard deviation divided by the square root of the episode count. P95 is the nearest-rank 95th percentile. Means include capped failures if present; here there are none. CSV output additionally includes success rates, standard deviations, normal-approximation 95% intervals, blocked attempts and slips.

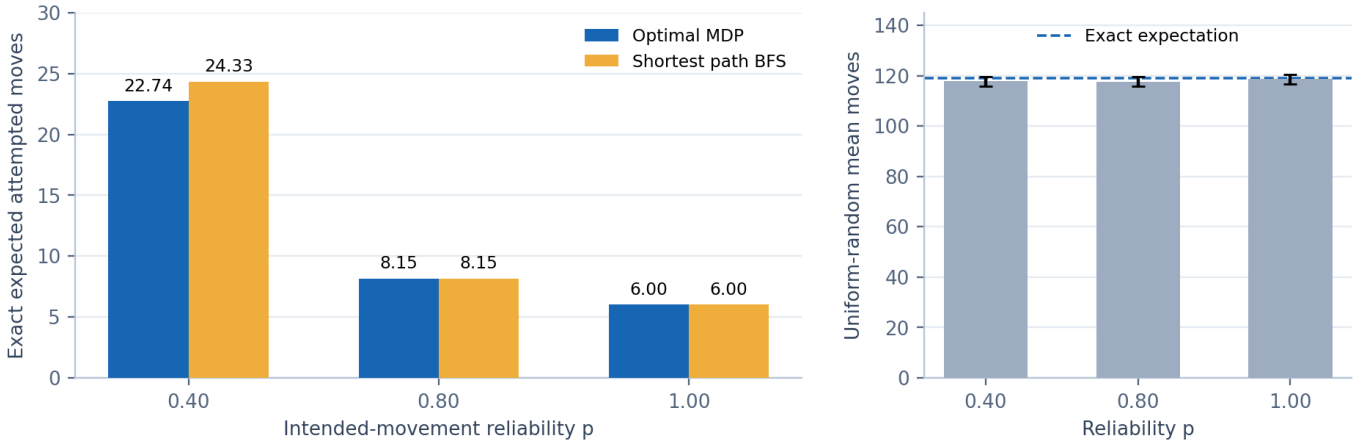


Figure 2. Left: exact expected costs for the two directed controllers. Right: random-controller sample means with approximate 95% intervals and its exact expectation. Separate vertical scales keep both comparisons readable.

At p=1.00, both directed controllers recover the six-edge BFS route. At p=0.80, their expected costs are equal at 8.1465, and the measured mean is 8.1854. At p=0.40, the optimal expected cost is 22.7396 compared with 24.3281 for BFS: a 6.53% reduction. No improvement over BFS is claimed in the other two settings.

The random policy has exact expectation 118.9091 at every tested p: uniform relative actions imply uniform attempted compass bearings regardless of the motion reliability. Monte Carlo means vary with the sampled trajectories. At p=0, BFS from Green Park is improper and has infinite uncapped expected cost; the application represents this explicitly.

4. Verification and demonstration

The delivered build passes **10685 assertions** using Java 17.0.20 in a Linux environment. The actual Swing panel is also rendered offscreen for layout inspection. A native Windows desktop session has not been executed here; the Windows scripts and VS Code configuration are included for the user to run locally.

Check	Evidence
Map and actions	14 stations, 15 roads, four source-map chains, connected goal and inverse relative turns.
Transition model	Every state/action at p=0, 0.01, 0.40, 0.80, 1 has normalized probabilities; movement follows valid roads or stays.
Boundary and goal	Blocked turns, curved-road arrival heading, absorbing goal, starting at goal and truncation behavior.
Deterministic optimum	All 56 states agree with independent BFS distances at p=1; rollouts waste no attempts.
Stochastic optimum	At p=0, 0.20, 0.40, 0.80, 1, policy values agree with independent linear equations and dominate both baselines.
Sampling and reproducibility	100,000 motion samples match configured probabilities; repeated seeded traces agree; a 10,000-episode mean agrees with exact cost.
Actual Swing controls	Step buttons reach the goal and disable further movement; reset restores controls; starting at the goal takes zero moves.

Run in VS Code on Windows

Extract the ZIP and open the **london-agent-java** folder. Use JDK 17 or newer and Extension Pack for Java. In a new PowerShell terminal:

```
java -version
javac -version
.\run.bat
.\test.bat
```

The included JAR also starts with `java -jar dist/london-agent.jar`. The project has no external Java dependencies. On Linux/macOS use `bash run.sh`.

Presentation sequence

1. Show the map and explain station + heading.
2. Set p=1.00 and demonstrate the six-step route.
3. Reset at p=0.80, seed 42, and show the step-6 slip and recovery.
4. Compare policies at p=0.40 and distinguish empirical means from exact expectations.
5. Show the policy table and the transition/Bellman code. The English script and Persian run guide are included in the project folder.

Scope and references

The results concern the reconstructed teaching schematic and declared motion model. They do not estimate real London travel time, congestion or transport routing. Road ports, probabilities and unit attempt costs should be reviewed if the teacher supplies a different convention.

[1] Supplied assignment. The motion of the agent in London, file 3. Travel in London.pdf. A copy is included under assignment/.
[2] Bertsekas, D. P., and Tsitsiklis, J. N. (1991). An Analysis of Stochastic Shortest Path Problems. Mathematics of Operations Research, 16(3), 580-595. doi:10.1287/moor.16.3.580. Author-hosted PDF.
[3] Oracle. Java SE 17 Swing API and threading policy. UI operations use the event dispatch thread; batch comparison uses SwingWorker.